Experiment No : 04

Aim :

Learn use of maplotlib library as visualisation tool in data science pipeline to create contour plots, histograms and box plots.

Performance :

[Students need to execute each and every cell in this section and note the output of the same. Once done they have to answer Questions mentioned in review section]

Density and Contour plots

Sometimes it is useful to display three-dimensional data in two dimensions using contours or color-coded regions.

There are three Matplotlib functions that can be helpful for this task: plt.contour for contour plots, plt.contourf for filled contour plots, and plt.imshow for showing images.

We will see examples of this type of plot in section below

```
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-white')
import numpy as np
```

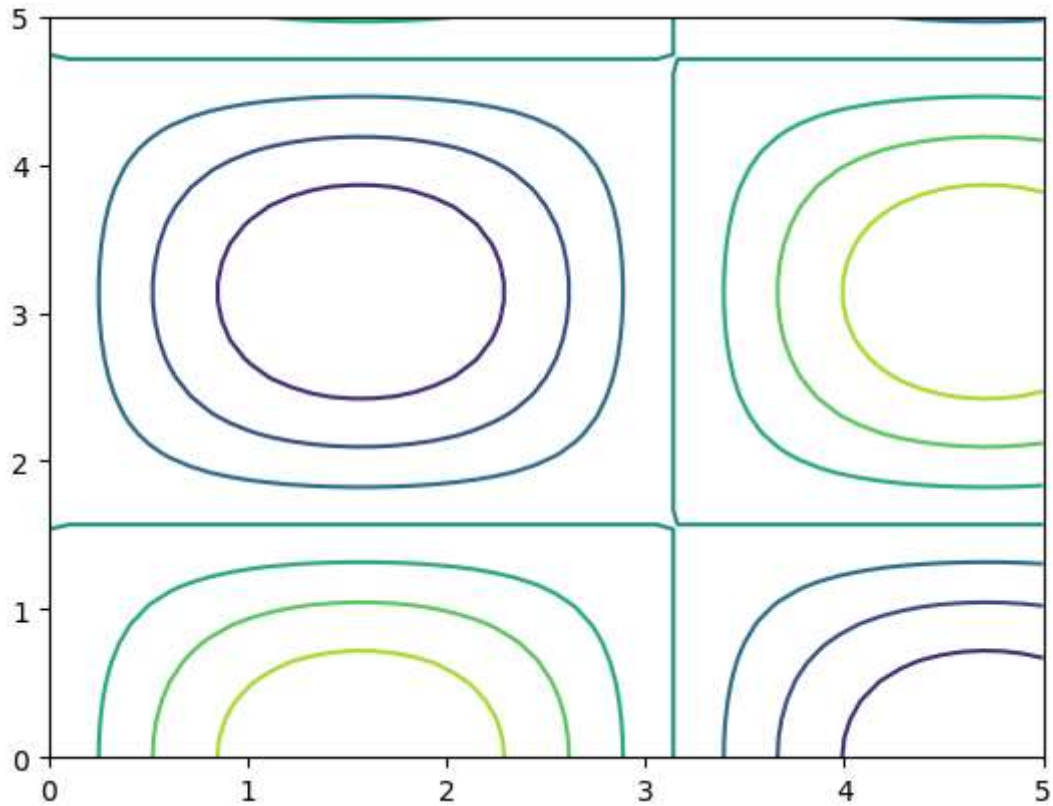
```
# A 3D function to plot

def f(x, y):
    return np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)
```

A contour plot can be created with the plt.contour function. It takes three arguments: a grid of x values, a grid of y values, and a grid of z values.

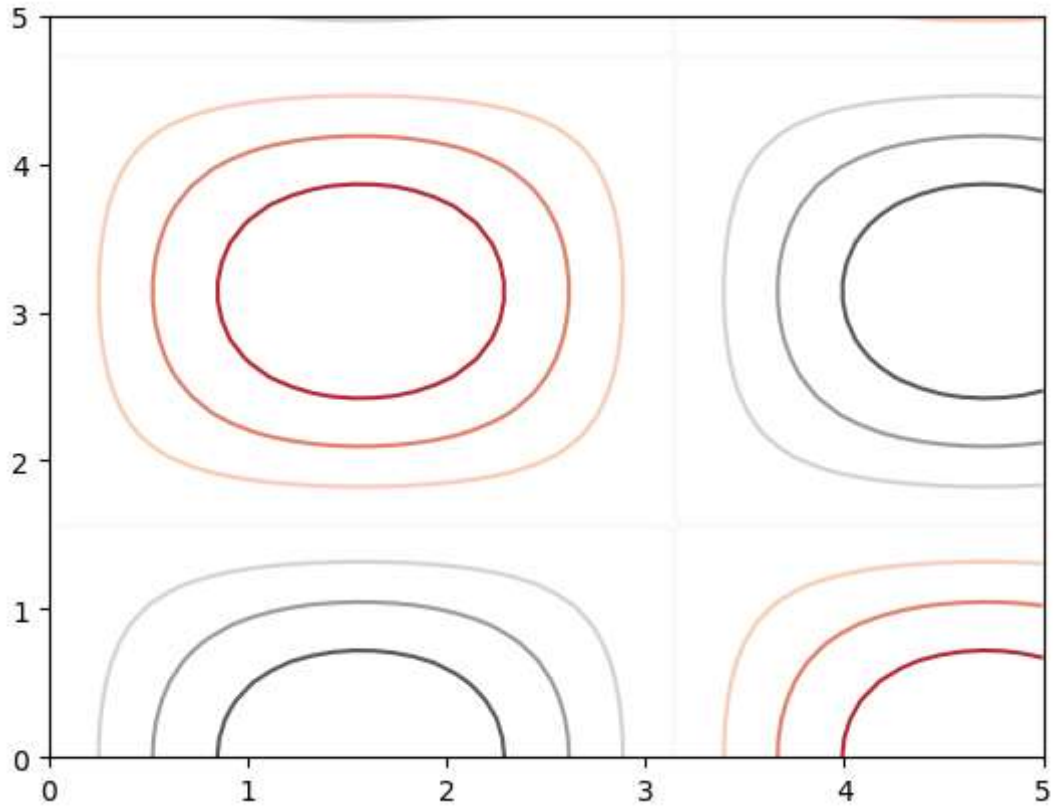
The x and y values represent positions on the plot, and the z values will be represented by the contour levels.

Perhaps the most straightforward way to prepare such data is to use the np.meshgrid function, which builds two-dimensional grids from one-dimensional arrays:

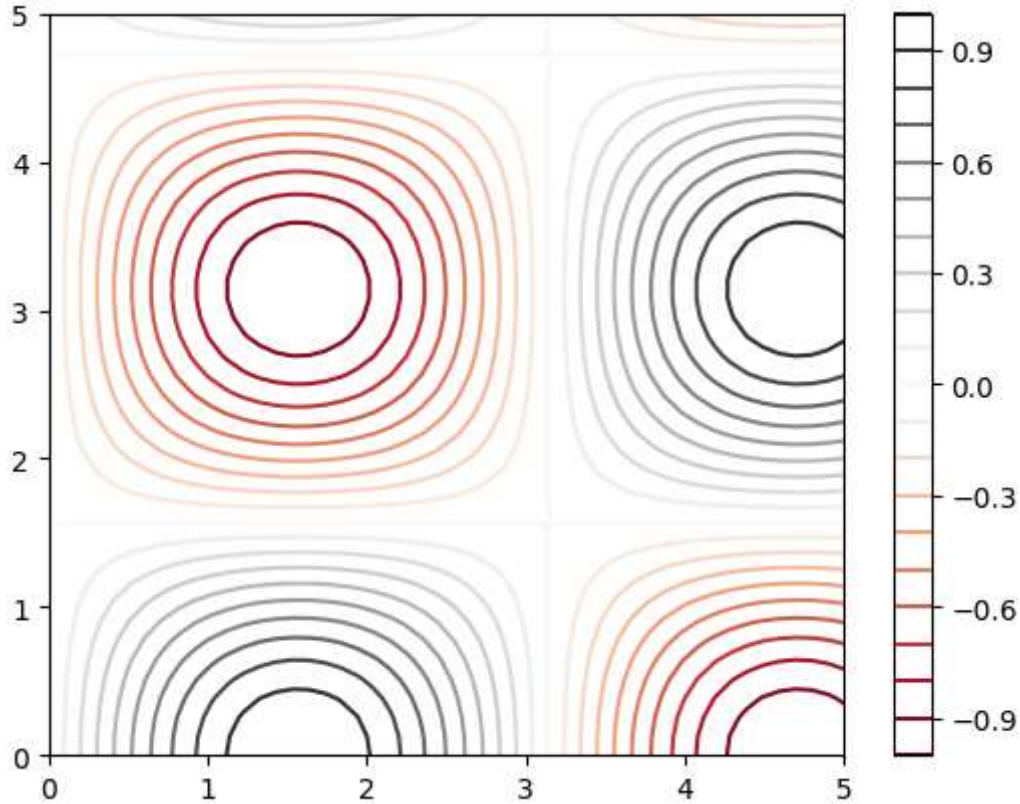
```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

# define function
def f(x, y):
    return np.sin(x) * np.cos(y)

# x is set of 50 values between 0 and 5
x = np.linspace(0, 5, 50)

# y is set of 40 values between 0 and 5
y = np.linspace(0, 5, 40)

X, Y = np.meshgrid(x, y)

Z = f(X, Y)

plt.contour(X, Y, Z)
plt.show()
```




```
plt.contour(X, Y, Z, cmap='RdGy');
```




```
plt.contour(X, Y, Z, 20, cmap='RdGy');
plt.colorbar();
```



1 :- Explain what are the changes you observe in contour plots above 3 code cells ?

Ans :- 1:- Change in Number of Grid Points

If the values in np.linspace() increase (for example from 20 → 40 → 50 points), the grid becomes denser.

More grid points mean more data points for the function.

2 :- Smoothness of the Contour Lines

With fewer points, contour lines look rough or blocky.

With more points, the contour lines become smoother and clearer.

3 :- Detail of the Plot

Low resolution → fewer contour levels visible.

High resolution → more detailed pattern of the function.

2 :- What is size of X, Y and Z respectively ?

Ans :- In the given code, x contains 50 values and y contains 40 values generated using np.linspace().

When the np.meshgrid(x, y) function is applied, it creates two 2-dimensional arrays X and Y representing the coordinate grid. The number of rows corresponds to the number of values in y (40) and the number of columns corresponds to the number of values in x (50).

Therefore:

Size of X = (40 × 50)

Size of Y = (40 × 50)

Since Z = f(X, Y) and the function is applied element-wise to the arrays X and Y, the resulting matrix Z also has the same dimensions.

Hence, the size of X, Y, and Z is (40 × 50).

3 :- How does colorbar() method help in contour plot.c

Ans :- The colorbar() method in a contour plot is used to display a color scale that represents the values of the function plotted in the graph. It helps the viewer understand

the relationship between the colors in the contour plot and the corresponding numerical values of the data.

In a contour plot, different colors represent different ranges of values of the function Z. The colorbar() method adds a vertical or horizontal bar beside the plot, which shows the gradient of colors along with their corresponding numeric values. This makes it easier to interpret the contour levels.

By using the colorbar, users can identify which color corresponds to higher or lower values of the function. It improves the readability and understanding of the plot, especially when multiple contour levels or color variations are used.

Thus, the colorbar() method helps in visually interpreting the data values represented by colors in the contour plot.

Creating Smoother Contour Plots

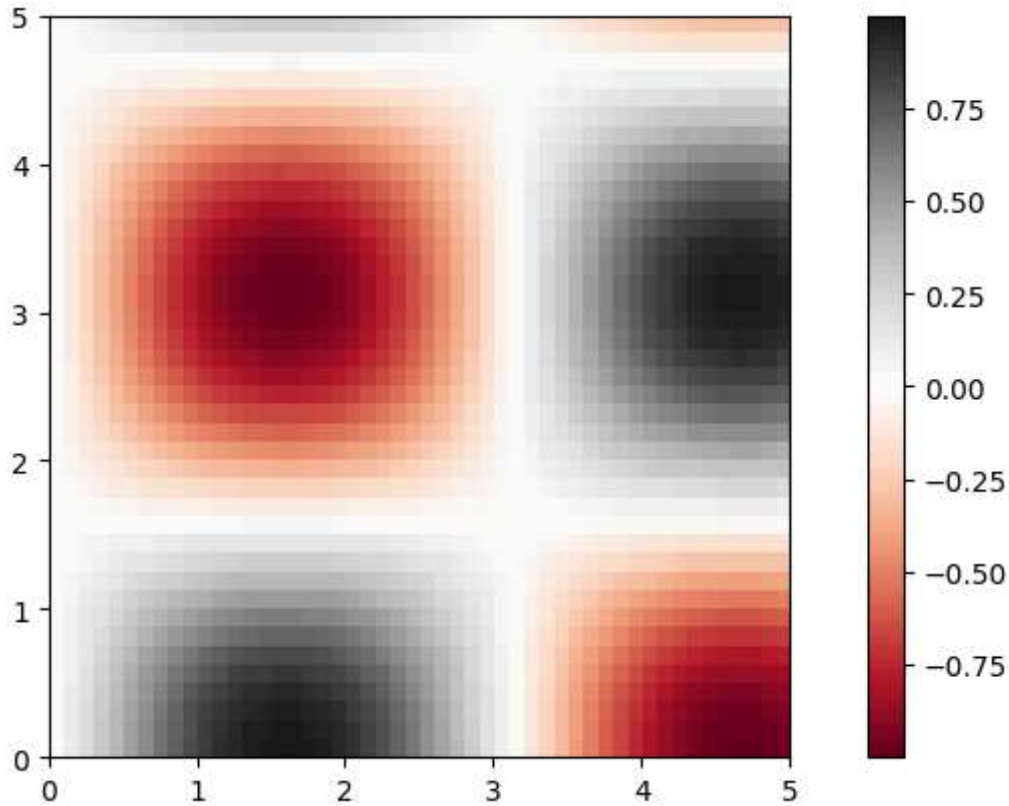
One potential issue with this plot is that it is a bit "splotchy." That is, the color steps are discrete rather than continuous, which is not always what is desired.

This could be remedied by setting the number of contours to a very high number, but this results in a rather inefficient plot: Matplotlib must render a new polygon for each step in the level. A better way to handle this is to use the plt.imshow() function, which interprets a two-dimensional grid of data as an image.

The following code shows how to perform this:

```
plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower', cmap='RdGy')
plt.colorbar()
plt.axis('image')
```

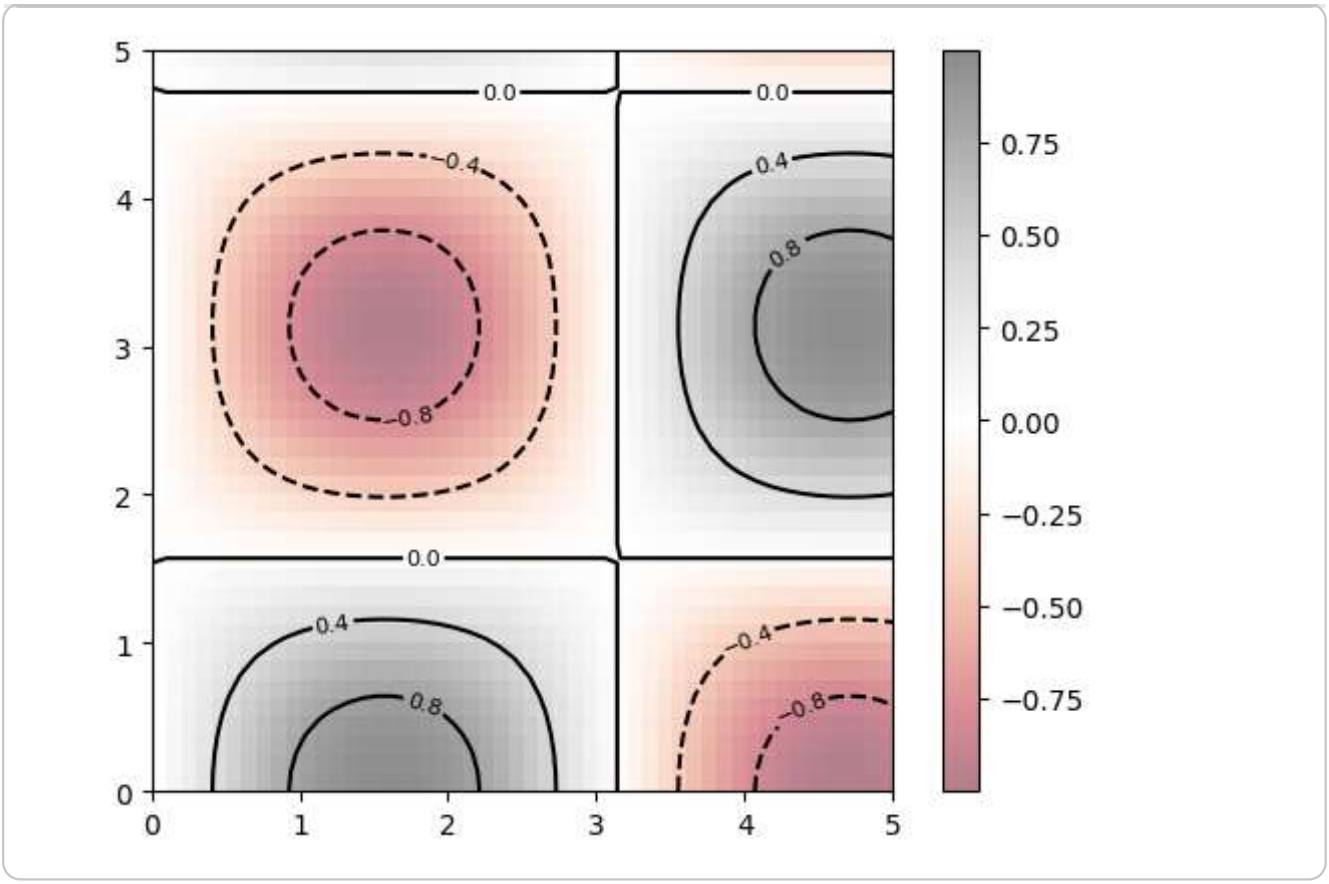
```
(np.float64(0.0), np.float64(5.0), np.float64(0.0), np.float64(5.0))
```



Things to remember while using imshow():

plt.imshow() doesn't accept an x and y grid, so you must manually specify the extent [xmin, xmax, ymin, ymax] of the image on the plot. plt.imshow() by default follows the standard image array definition where the origin is in the upper left, not in the lower left as in most contour plots. This must be changed when showing gridded data. plt.imshow() will automatically adjust the axis aspect ratio to match the input data; this can be changed by setting, for example, plt.axis(aspect='image') to make x and y units match.

```
contours = plt.contour(X, Y, Z, 5, colors='black')
plt.clabel(contours, inline=True, fontsize=8)

plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower',
           cmap='RdGy', alpha=0.5)
plt.colorbar();
```



Answer Following Questions :

1 :- What does alpha parameter do ?

Ans :- The alpha parameter in Matplotlib is used to control the transparency (opacity) of a plot element such as lines, markers, images, or filled regions.

The value of alpha ranges from 0 to 1:

alpha = 0 ⟶ completely transparent (invisible)

alpha = 1 ⟶ completely opaque (fully visible)

By adjusting the alpha value, we can make objects in the plot partially transparent, which is useful when multiple plots overlap. It helps to visualize overlapping data clearly and improves the readability of the graph.

For example, if alpha=0.5 is used, the plotted object will appear 50% transparent, allowing other elements behind it to be seen.

Thus, the alpha parameter controls the transparency level of graphical elements in a plot.

2 :- What does clabel method does ?

Ans :- The clabel() method in Matplotlib is used to add labels to the contour lines in a contour plot. It displays the numerical values of the contour levels directly on the contour lines, making the plot easier to interpret.

When a contour plot is created using the contour() function, it only shows lines representing different values of the function. By using the clabel() method, these lines are labeled with their corresponding data values, helping the viewer understand what each contour line represents.

This method improves the readability and clarity of the contour plot, especially when multiple contour levels are present.

Thus, the clabel() method is used to display the values of contour levels on the contour lines in a contour plot.

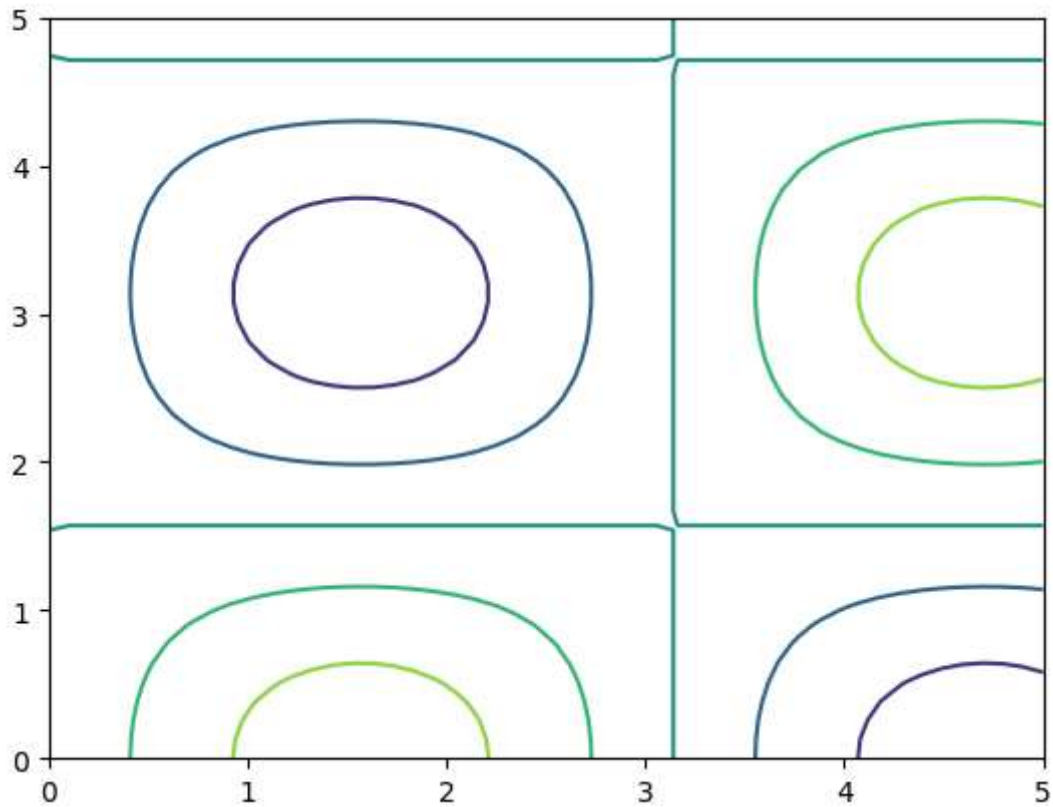
3 :- Recreate above code block after, changing parameter 4 in contour fuction to 5 from 3. Explain what change you see in the plot.

To recreate the code block, the fourth parameter in the contour() function is changed from 3 to 5.

Modified Code

```
plt.contour(X, Y, Z, 5)
plt.clabel()
plt.show()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_383/1093606729.py in <cell line: 0>()
      1 plt.contour(X, Y, Z, 5)
----> 2 plt.clabel()
      3 plt.show()

TypeError: clabel() missing 1 required positional argument: 'CS'
```



Explanation of Change

Earlier, the value 3 in the contour() function meant that the plot displayed 3 contour levels. After changing this value to 5, the plot now displays 5 contour levels.

Because of this change:

More contour lines appear in the plot.

The plot shows more detailed variation of the function values.

The representation of the surface becomes clearer and more informative.

Conclusion: Increasing the parameter from 3 to 5 increases the number of contour levels, resulting in more contour lines and a more detailed contour plot.

Histogram, Binning Plots

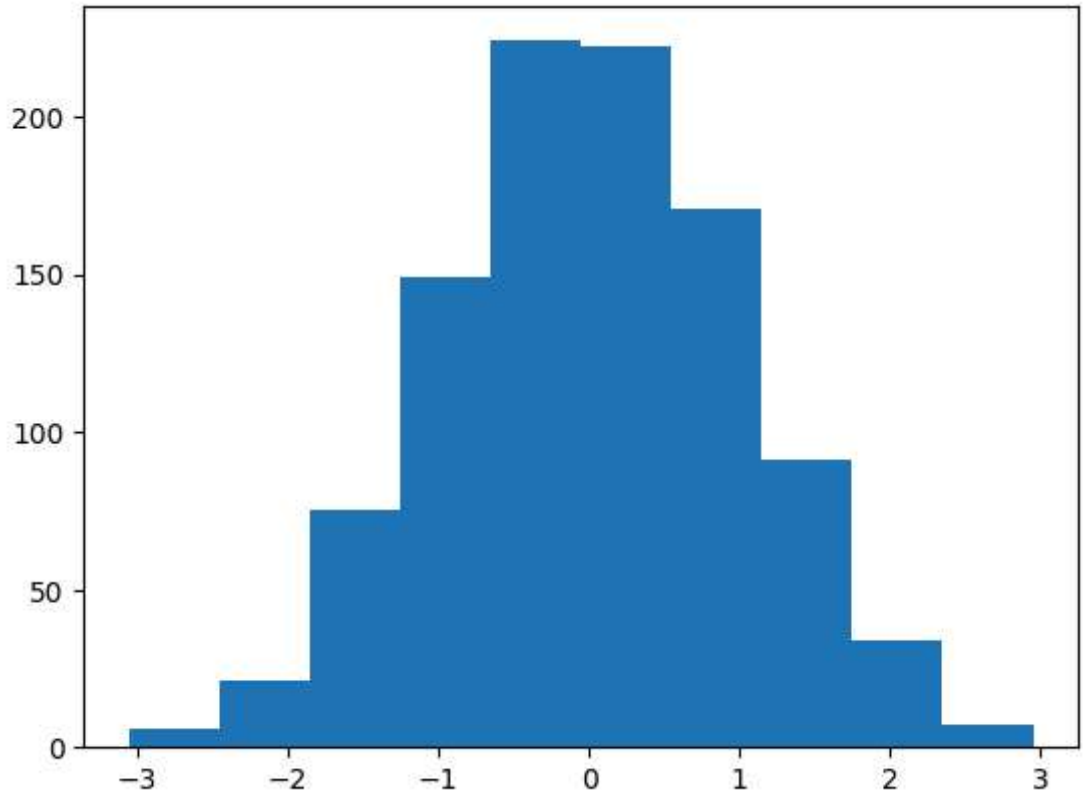
A histogram is a graphical representation of a grouped frequency distribution with continuous classes. It is an area diagram and can be defined as a set of rectangles with bases along with the intervals between class boundaries and with areas proportional to frequencies in the corresponding classes. In such representations, all the rectangles are adjacent since the base covers the intervals between class boundaries. The heights of rectangles are proportional to corresponding frequencies of similar classes and for different classes, the heights will be proportional to corresponding frequency densities.

In other words, a histogram is a diagram involving rectangles whose area is proportional to the frequency of a variable and width is equal to the class interval.

The histogram graph is used under certain conditions. They are:

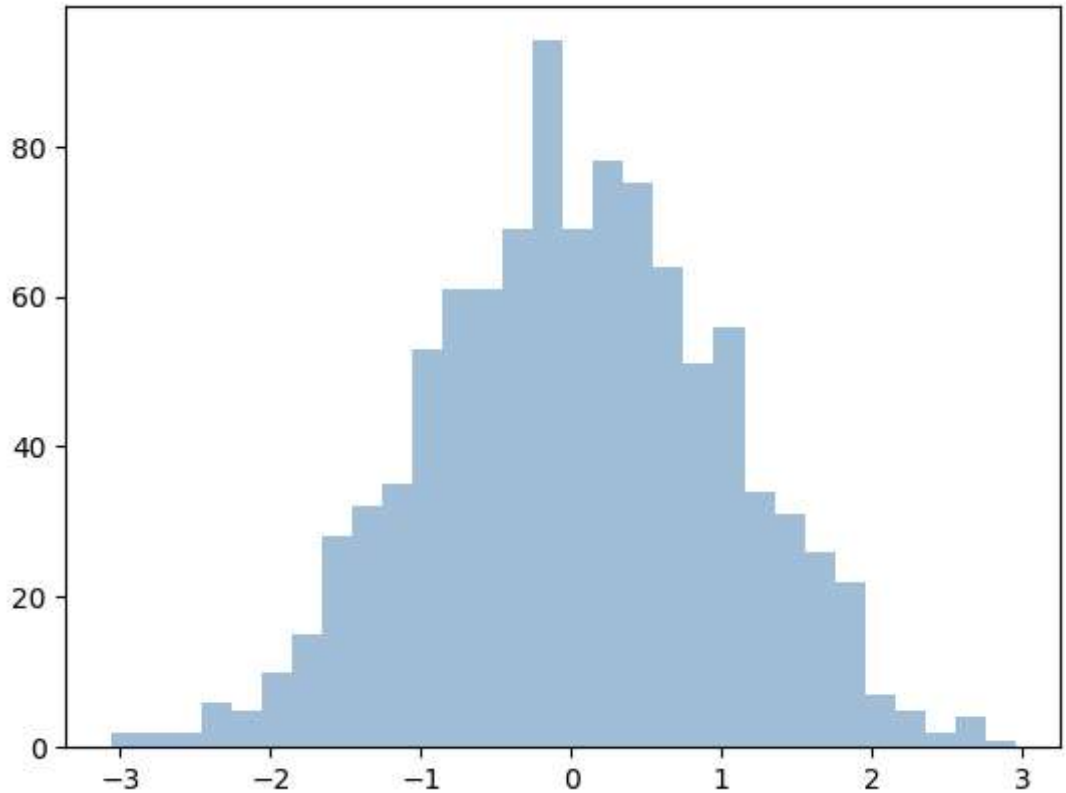
The data should be numerical. A histogram is used to check the shape of the data distribution. Used to check whether the process changes from one period to another. Used to determine whether the output is different when it involves two or more processes. Used to analyse whether the given process meets the customer requirements.

```
# How to plot histogram
data = np.random.randn(1000)
plt.hist(data);
```



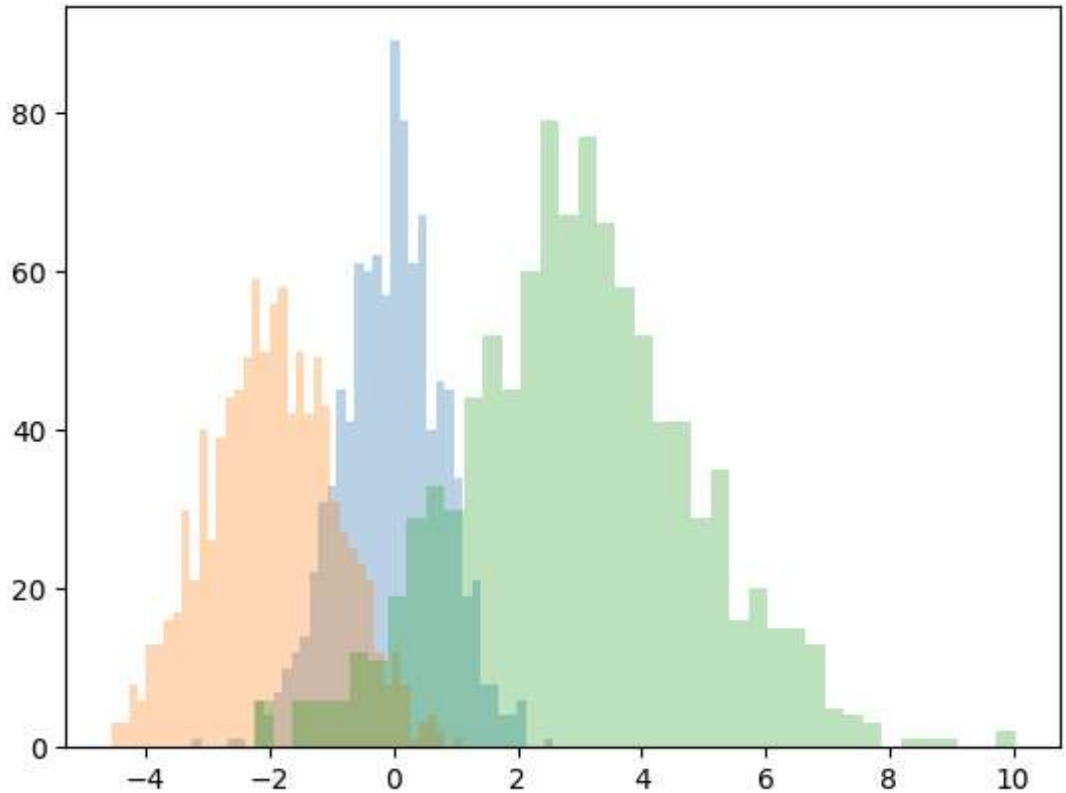
The hist() function has many options to tune both the calculation and the display; here's an example of a more customized histogram:

```
plt.hist(data, bins=30, alpha=0.5, histtype='stepfilled', color='steelblue', ec
```



The plt.hist docstring has more information on other customization options available. I find this combination of histtype='stepfilled' along with some transparency alpha to be very useful when comparing histograms of several distributions:

```
x1 = np.random.normal(0, 0.8, 1000)
x2 = np.random.normal(-2, 1, 1000)
x3 = np.random.normal(3, 2, 1000)

kwargs = dict(histtype='stepfilled', alpha=0.3, bins=40)

plt.hist(x1, **kwargs)
plt.hist(x2, **kwargs)
plt.hist(x3, **kwargs);
```



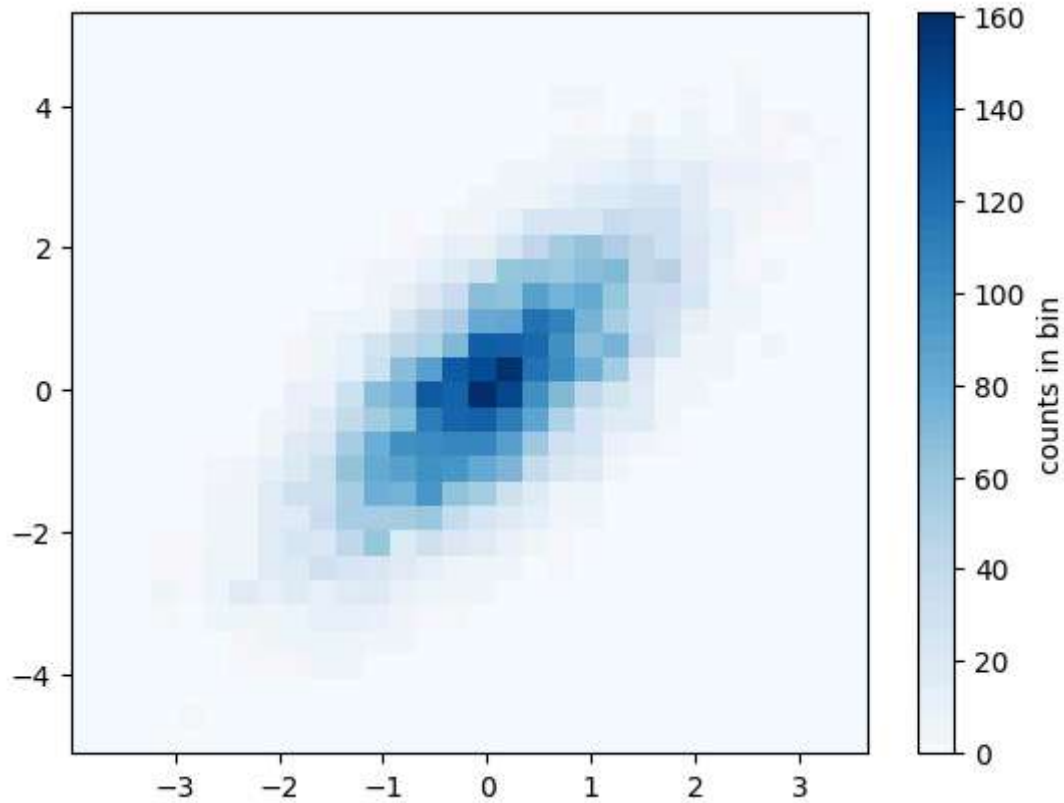

```
#Only storing histogram bin frequencies without ploting it
counts, bin_edges = np.histogram(data, bins=5)
print(counts,bin_edges)
```

```
[ 27 224 446 262  41] [-3.05964808 -1.85676961 -0.65389114  0.54898733  1.751865
```

plt.hist2d: Two-dimensional histogram

```
# Create 2 Variable normally distributed data sets with 10000
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 10000).T
```

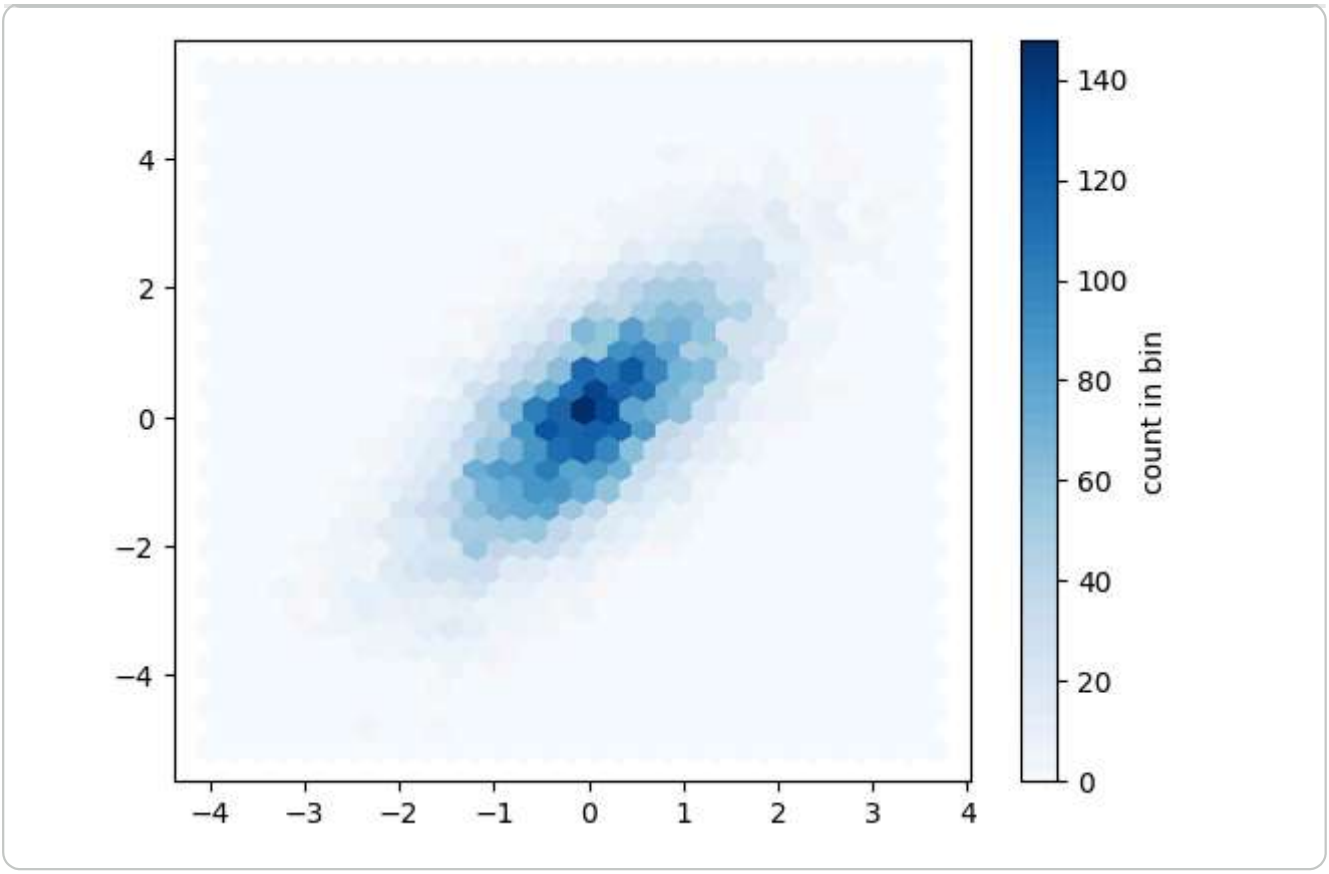
```
#Using 2d Histogram plot
plt.hist2d(x, y, bins=30, cmap='Blues')
cb = plt.colorbar()
cb.set_label('counts in bin')
```



plt.hexbin: Hexagonal binnings

The two-dimensional histogram creates a tesselation of squares across the axes. Another natural shape for such a tesselation is the regular hexagon. For this purpose, Matplotlib provides the plt.hexbin routine, which will represents a two-dimensional dataset binned within a grid of hexagons:

```
plt.hexbin(x, y, gridsize=30, cmap='Blues')
cb = plt.colorbar(label='count in bin')
```



1 : - What is role of bins parameter in histogram plot?

Ans :- The bins parameter in a histogram plot is used to divide the data into a number of intervals or groups. These intervals are called bins.

The bins parameter determines how many bars the histogram will have and how the data values are grouped together. Each bin represents a range of values, and the height of the bar shows how many data points fall within that range.

If the number of bins is small, the histogram will have fewer bars and the data distribution may appear less detailed. If the number of bins is large, the histogram will have more bars, showing a more detailed distribution of the data.

Thus, the bins parameter controls the number of intervals used to group the data in a histogram, which affects the level of detail in the plot.

2 :- Create a set of 10000 uniform random samples and store it in variable named norm_data. Draw histograms of norm_data with following bin counts 5, 50, 500, 5000.

```
import numpy as np
import matplotlib.pyplot as plt

# Create 10000 uniform random samples
```

```
norm_data = np.random.rand(10000)

# Histogram with 5 bins
plt.hist(norm_data, bins=5)
plt.title("Histogram with 5 bins")
plt.show()

# Histogram with 50 bins
plt.hist(norm_data, bins=50)
plt.title("Histogram with 50 bins")
plt.show()

# Histogram with 500 bins
plt.hist(norm_data, bins=500)
plt.title("Histogram with 500 bins")
plt.show()

# Histogram with 5000 bins
plt.hist(norm_data, bins=5000)
plt.title("Histogram with 5000 bins")
plt.show()
```

Explanation:

np.random.rand(10000) generates 10000 uniform random samples between 0 and 1.

The plt.hist() function is used to draw the histogram.

The bins parameter controls the number of intervals used to group the data.

As the number of bins increases (5 → 50 → 500 → 5000), the histogram shows more detailed distribution of the data.

3 : - Explain your observations in text box of above step.

Ans :- When the histogram is drawn with different bin counts, the appearance of the plot changes depending on how the data is grouped.

When 5 bins are used, the histogram has very few bars, so the data distribution appears very simple and less detailed. It only gives a rough idea of how the data is spread.

When 50 bins are used, the histogram becomes more detailed, and the distribution of the random data becomes clearer. The bars represent smaller intervals compared to the previous case.

When 500 bins are used, the histogram shows many narrow bars, which represent the data distribution in much finer detail. The variation in the random data becomes more visible.

When 5000 bins are used, the histogram becomes very dense with extremely thin bars. The plot may start to look noisy or cluttered, making it harder to interpret clearly.

Conclusion: As the number of bins increases, the histogram shows more detailed information about the data distribution, but too many bins can make the plot overly complex and difficult to read.

Ploting from a CSV File

Multiple Subplots

Sometimes it is helpful to compare different views of data side by side. Matplotlib has the concept of subplots : groups of smaller axes that can exist together within a single figure.

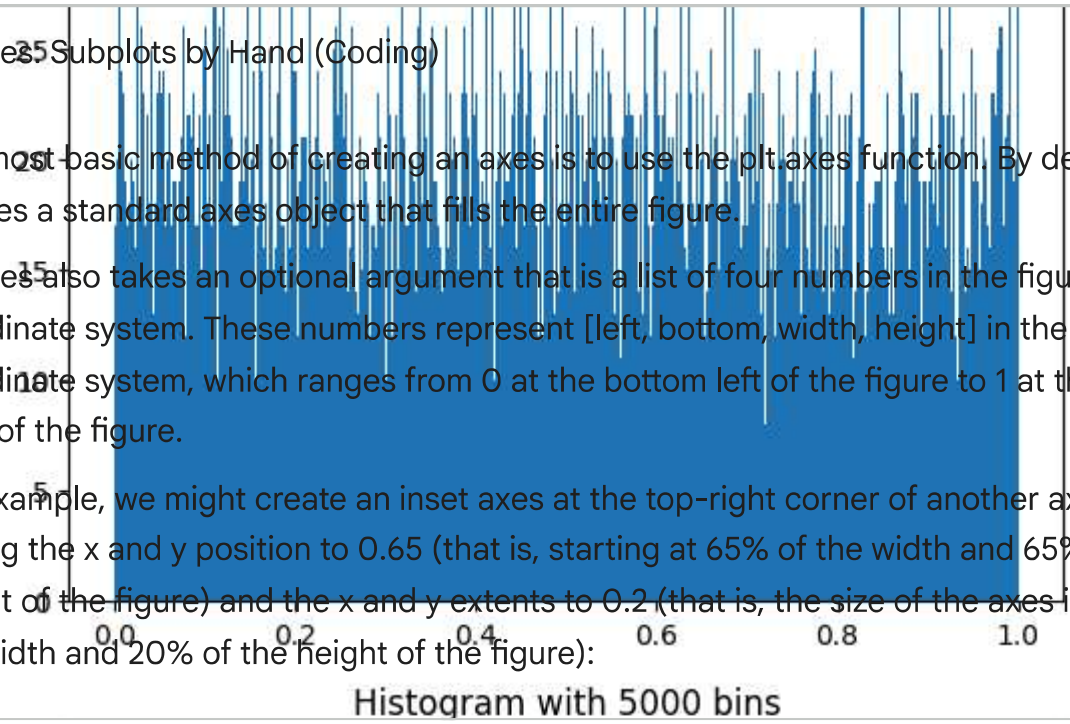
These subplots might be insets, grids of plots, or other more complicated layouts.

plt.axes: Subplots by Hand (Coding)

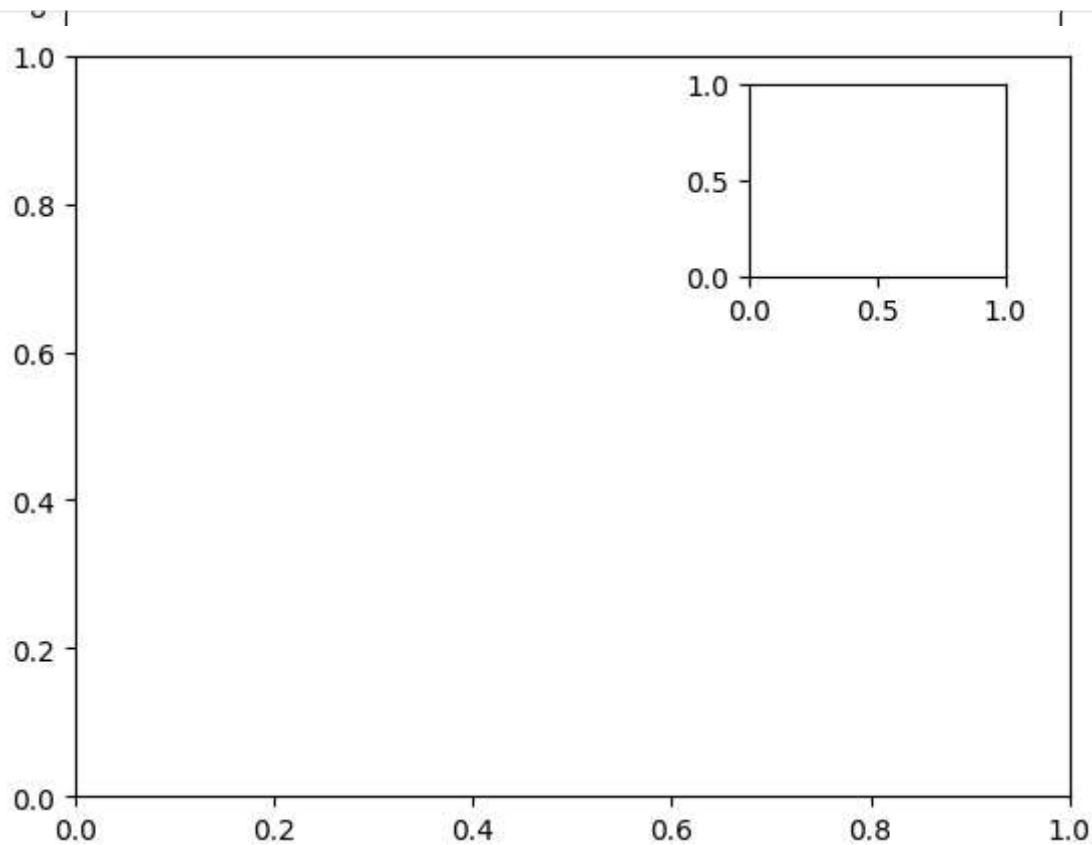
The most basic method of creating an axes is to use the plt.axes function. By default this creates a standard axes object that fills the entire figure.

plt.axes also takes an optional argument that is a list of four numbers in the figure coordinate system. These numbers represent [left, bottom, width, height] in the figure coordinate system, which ranges from 0 at the bottom left of the figure to 1 at the top right of the figure.

For example, we might create an inset axes at the top-right corner of another axes by setting the x and y position to 0.65 (that is, starting at 65% of the width and 65% of the height of the figure) and the x and y extents to 0.2 (that is, the size of the axes is 20% of the width and 20% of the height of the figure):

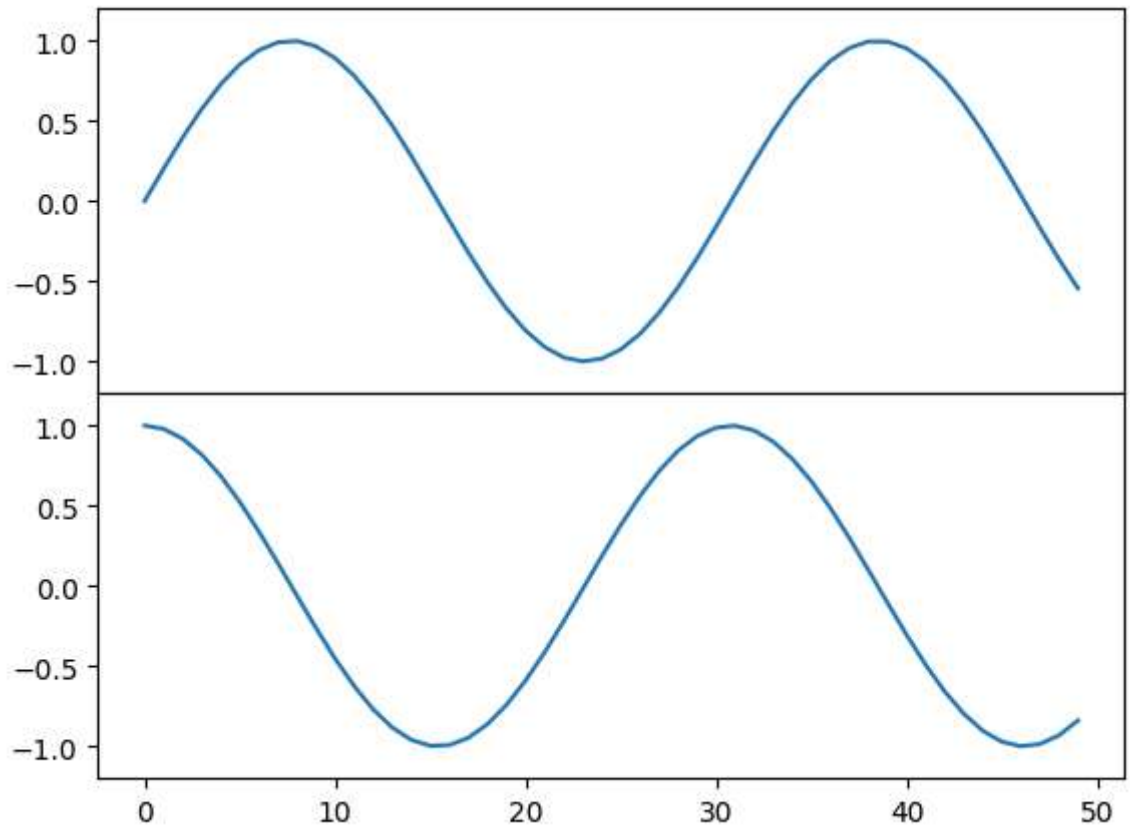



```
ax1 = plt.axes()  # standard axes
ax2 = plt.axes([0.650, 0.650, 0.2, 0.2]) #inset axes
```



```
#creating subplot using fig.add_axes
fig = plt.figure()
ax1 = fig.add_axes([0.1, 0.5, 0.8, 0.4],
```

```
                    xticklabels=[], ylim=(-1.2, 1.2))
ax2 = fig.add_axes([0.1, 0.1, 0.8, 0.4],
                    ylim=(-1.2, 1.2))

x = np.linspace(0, 10)
ax1.plot(np.sin(x))
ax2.plot(np.cos(x));
```



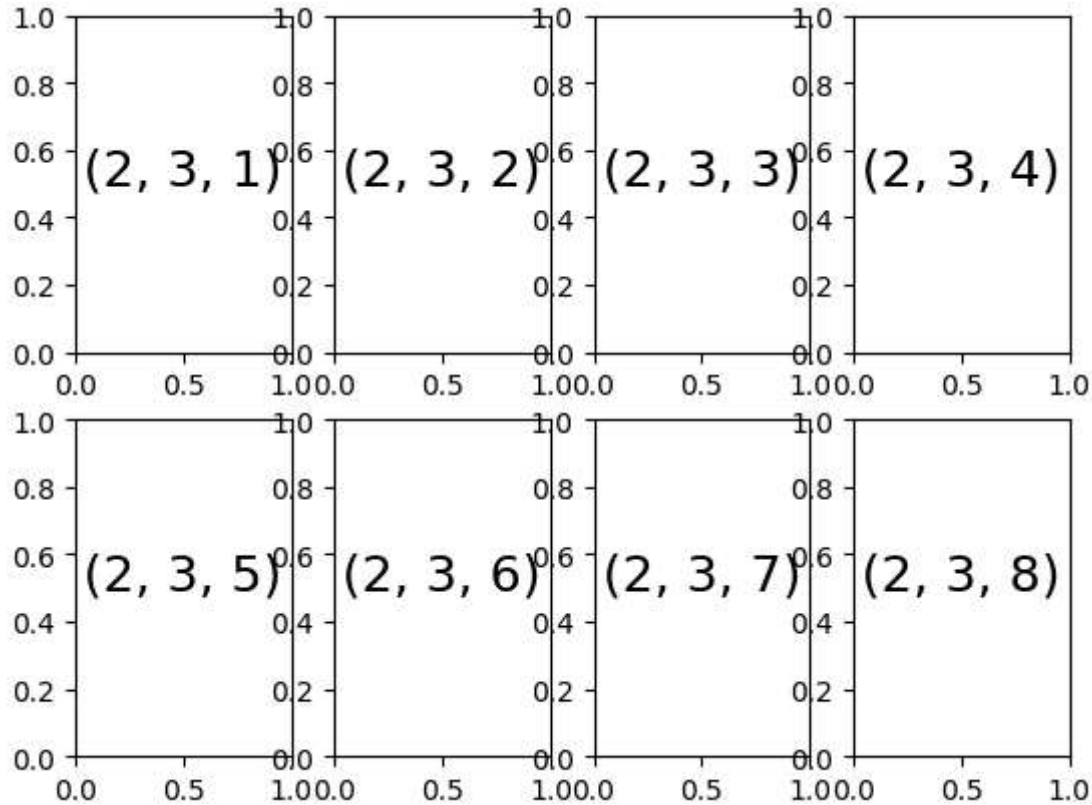
plt.subplot: Simple Grids of Subplots

Aligned columns or rows of subplots are a common-enough need that Matplotlib has several convenience routines that make them easy to create.

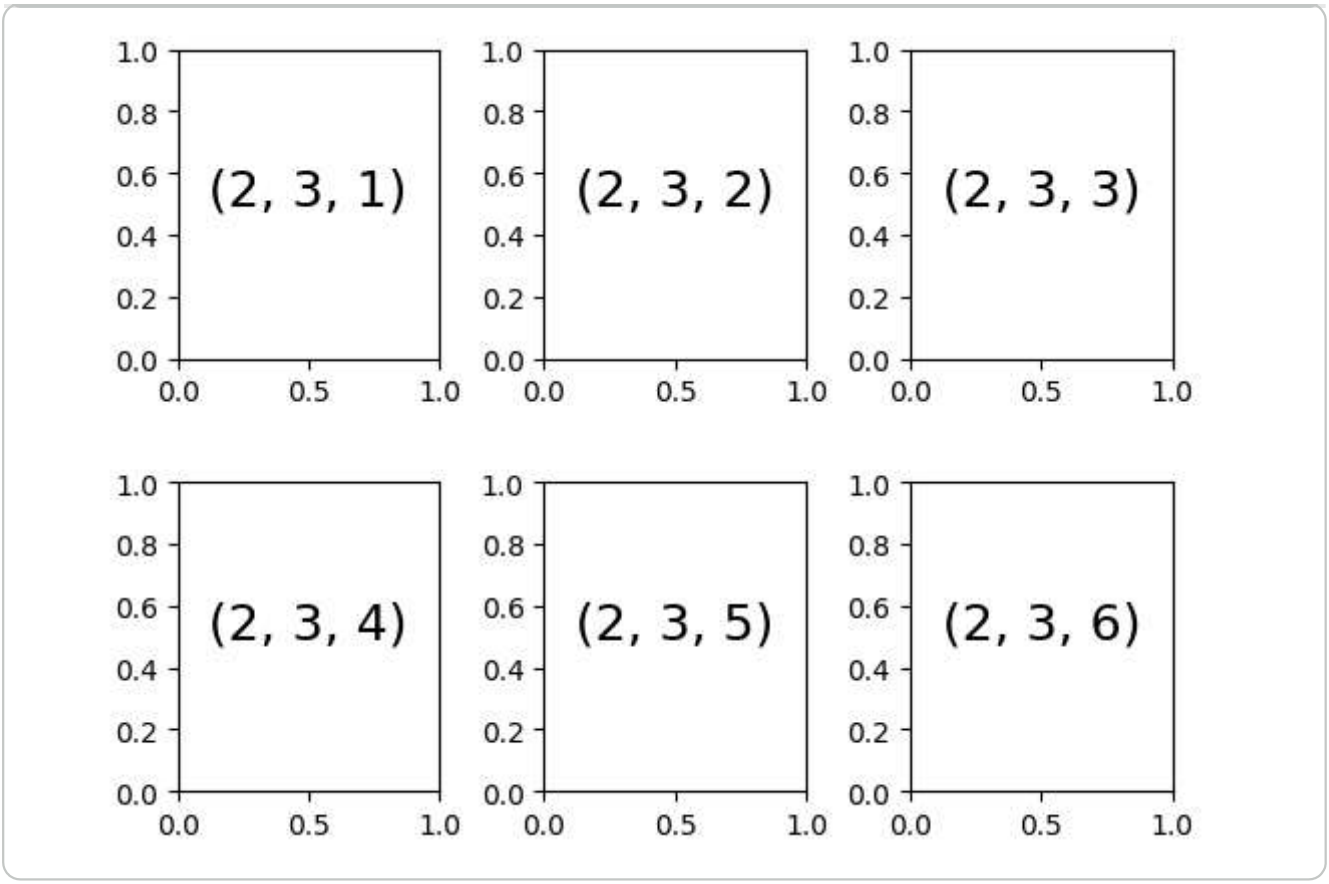
The lowest level of these is plt.subplot(), which creates a single subplot within a grid.

As you can see, this command takes three integer arguments—the number of rows, the number of columns, and the index of the plot to be created in this scheme, which runs from the upper left to the bottom right:

```
for i in range(1, 9):
    plt.subplot(2, 4, i)
    plt.text(0.5, 0.5, str((2, 3, i)),
             fontsize=18, ha='center')
```




```
fig = plt.figure()
fig.subplots_adjust(hspace=0.4, wspace=0.4)
for i in range(1, 7):
    ax = fig.add_subplot(2, 3, i)
    ax.text(0.5, 0.5, str((2, 3, i)),
            fontsize=18, ha='center')
```

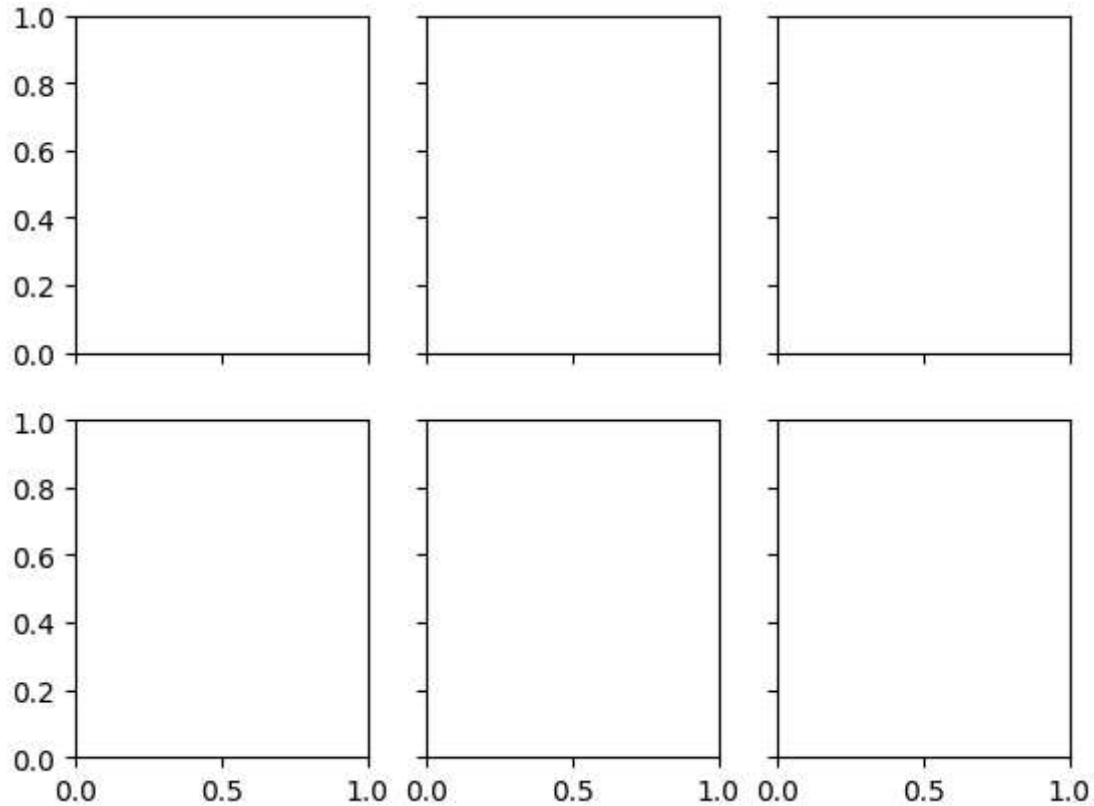


plt.subplots: The Whole Grid in One Go

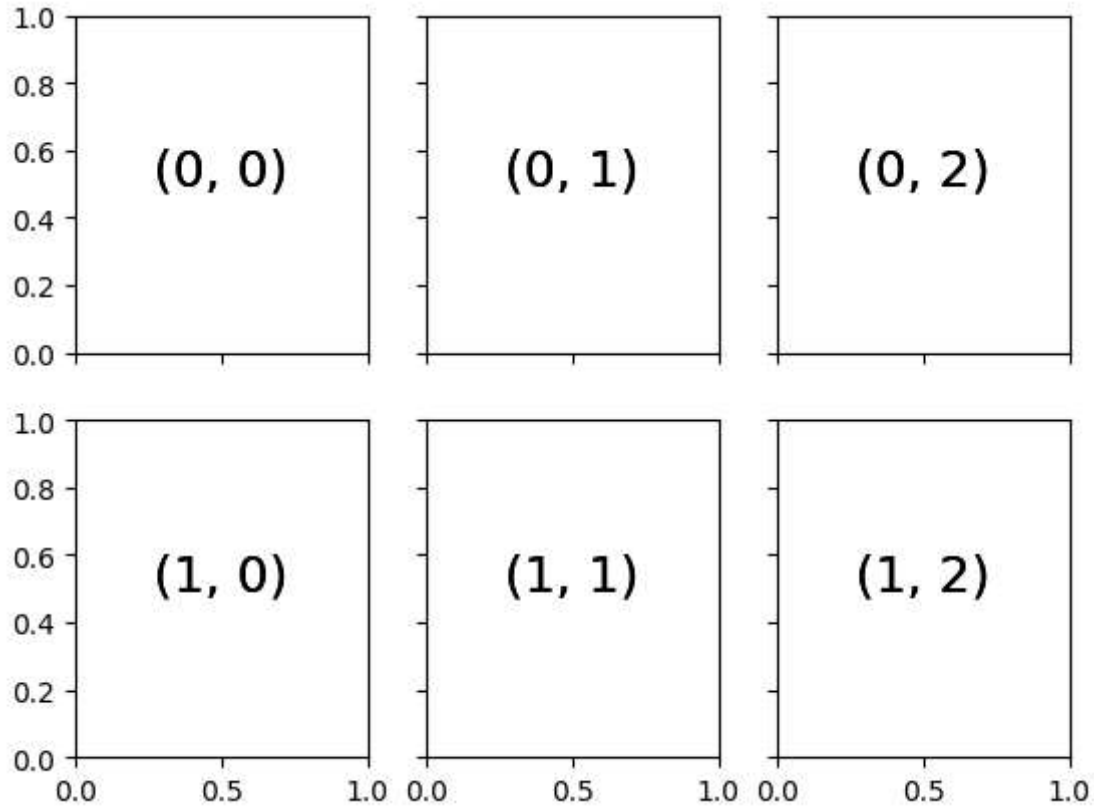
The approach just described can become quite tedious when creating a large grid of subplots, especially if you'd like to hide the x- and y-axis labels on the inner plots. For this purpose, plt.subplots() is the easier tool to use (note the s at the end of subplots). Rather than creating a single subplot, this function creates a full grid of subplots in a single line, returning them in a NumPy array. The arguments are the number of rows and number of columns, along with optional keywords sharex and sharey, which allow you to specify the relationships between different axes.

Here we'll create a grid of subplots, where all axes in the same row share their y-axis scale, and all axes in the same column share their x-axis scale:

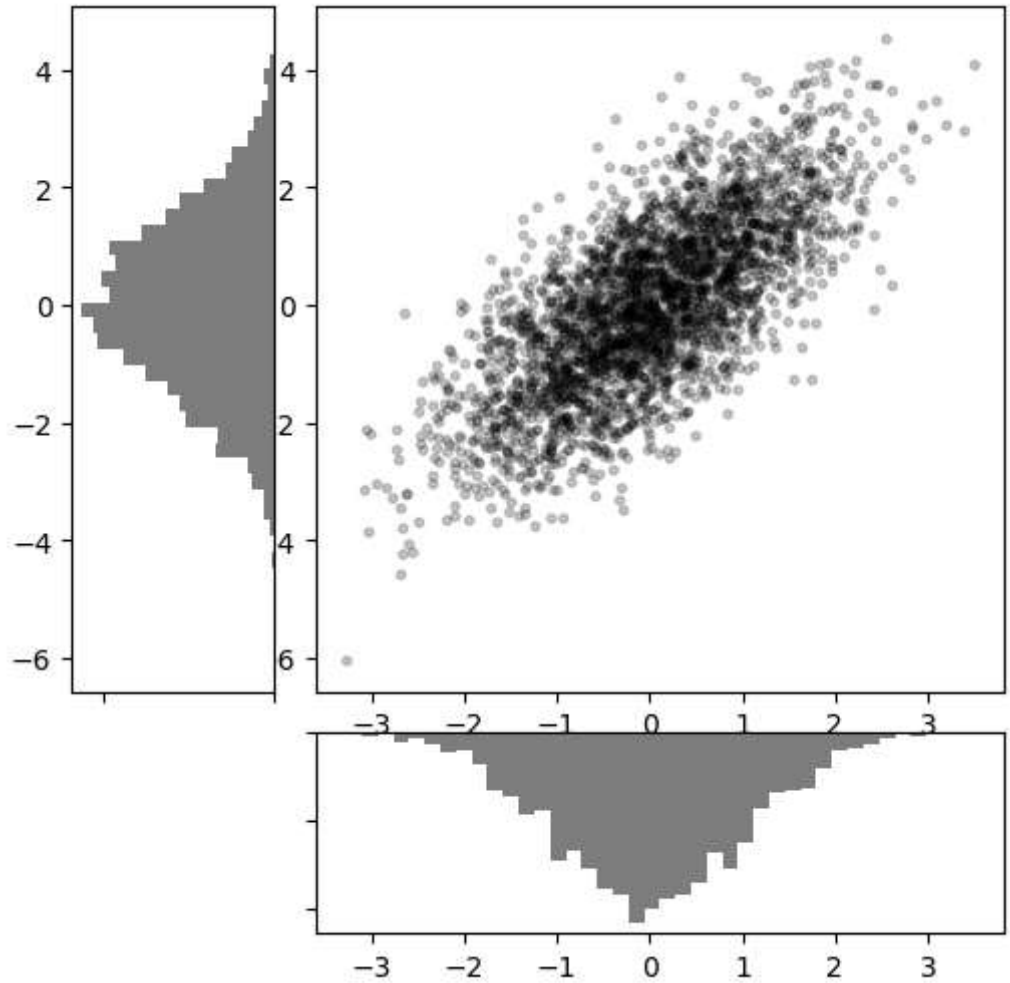
```
fig, ax = plt.subplots(2, 3, sharex='col', sharey='row')
```




```
# axes are in a two-dimensional array, indexed by [row, col]
for i in range(2):
    for j in range(3):
        ax[i, j].text(0.5, 0.5, str((i, j)),
                      fontsize=18, ha='center')
fig
```




```
# An example of Flexible grid arrangement
# Create some normally distributed data
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 3000).T

# Set up the axes with gridspec
fig = plt.figure(figsize=(6, 6))
grid = plt.GridSpec(4, 4, hspace=0.2, wspace=0.2)
main_ax = fig.add_subplot(grid[:-1, 1:])
y_hist = fig.add_subplot(grid[:-1, 0], xticklabels=[], sharey=main_ax)
x_hist = fig.add_subplot(grid[-1, 1:], yticklabels=[], sharex=main_ax)

# scatter points on the main axes
main_ax.plot(x, y, 'ok', markersize=3, alpha=0.2)

# histogram on the attached axes
x_hist.hist(x, 40, histtype='stepfilled',
            orientation='vertical', color='gray')
x_hist.invert_yaxis()

y_hist.hist(y, 40, histtype='stepfilled',
            orientation='horizontal', color='gray')
y_hist.invert_xaxis()
```




```
fig, ax = plt.subplots(figsize=(12, 4))
births_by_date.plot(ax=ax);
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_383/1795322712.py in <cell line: 0>()
      1 fig, ax = plt.subplots(figsize=(12, 4))
----> 2 births_by_date.plot(ax=ax);

NameError: name 'births_by_date' is not defined
```

```
Boxplot
```

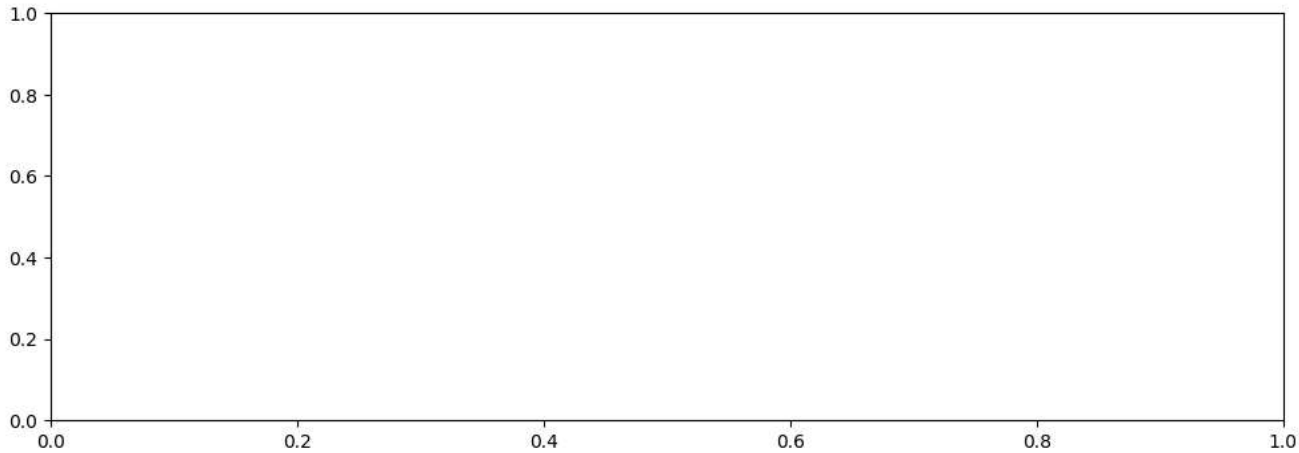
A Box Plot is also known as Whisker plot is created to display the summary of the set of data values having properties like minimum, first quartile, median, third quartile and maximum.

In the box plot, a box is created from the first quartile to the third quartile, a vertical line is also there which goes through the box at the median. Here x-axis denotes the data to be plotted while the y-axis shows the frequency distribution.

```
#Exmple of Box plot for some random data sets
# Import libraries
import matplotlib.pyplot as plt
import numpy as np


# Creating dataset
np.random.seed(10)
```

```
fig, ax = plt.subplots(figsize=(12, 4))
births_by_date.plot(ax=ax);
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_383/1795322712.py in <cell line: 0>()
      1 fig, ax = plt.subplots(figsize=(12, 4))
----> 2 births_by_date.plot(ax=ax);

NameError: name 'births_by_date' is not defined
```

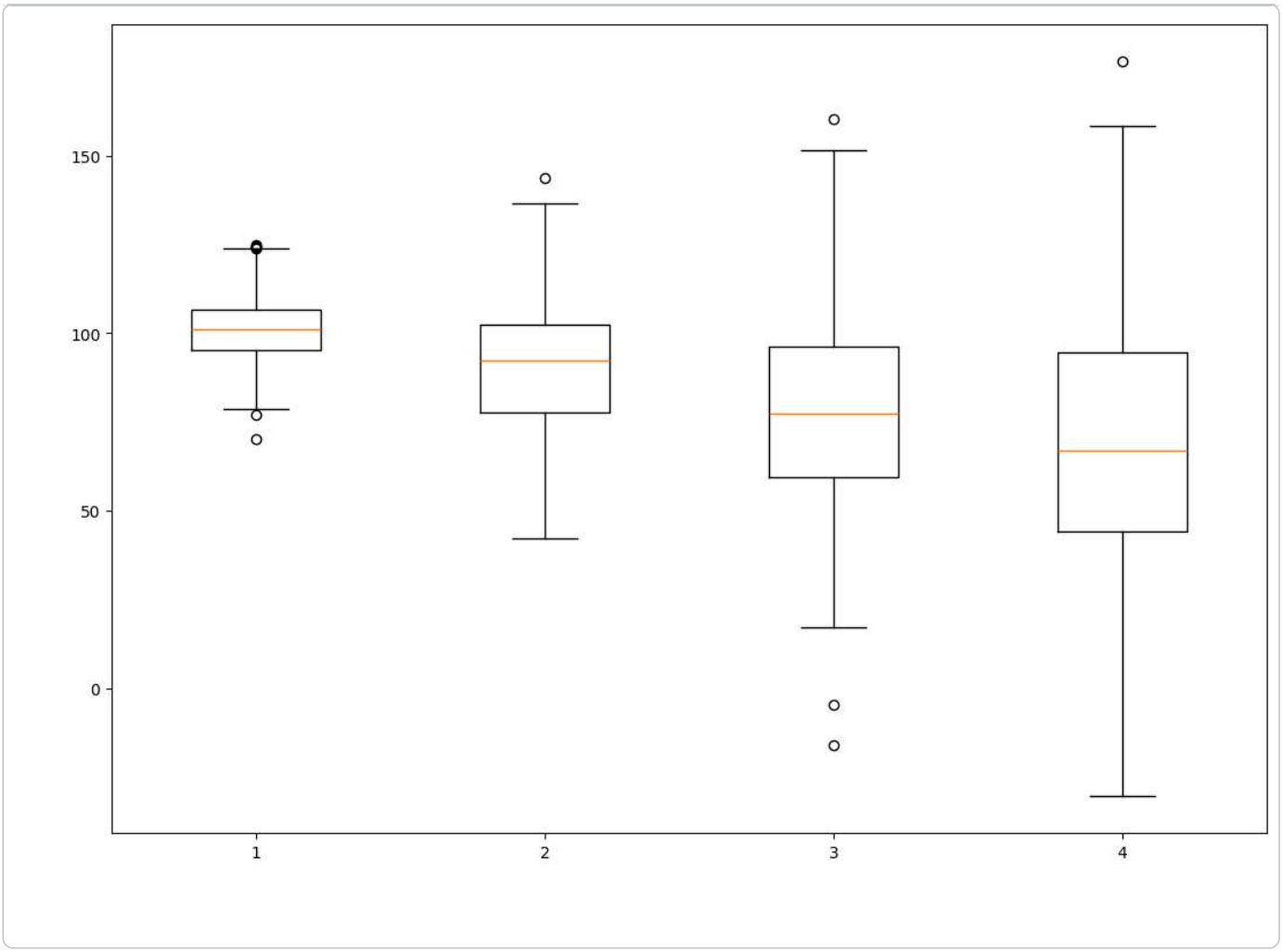



```
Boxplot
```

A Box Plot is also known as Whisker plot is created to display the summary of the set of data values having properties like minimum, first quartile, median, third quartile and maximum.

In the box plot, a box is created from the first quartile to the third quartile, a vertical line is also there which goes through the box at the median. Here x-axis denotes the data to be plotted while the y-axis shows the frequency distribution.

```
#Exmple of Box plot for some random data sets
# Import libraries
import matplotlib.pyplot as plt
import numpy as np


# Creating dataset
np.random.seed(10)

data_1 = np.random.normal(100, 10, 200)
data_2 = np.random.normal(90, 20, 200)
data_3 = np.random.normal(80, 30, 200)
data_4 = np.random.normal(70, 40, 200)
data = [data_1, data_2, data_3, data_4]

fig = plt.figure(figsize =(10, 7))

# Creating axes instance
ax = fig.add_axes([0, 0, 1, 1])

# Creating plot
bp = ax.boxplot(data)

# show plot
plt.show()
```



1 :- What is boxplot ?

Ans :- A boxplot (also called a box-and-whisker plot) is a graphical method used to display the distribution of a dataset based on five important summary statistics. It helps in understanding how the data is spread and identifying outliers.

A boxplot shows the following five values:

Minimum value – the smallest value in the dataset

First Quartile (Q1) – the value below which 25% of the data lies

Median (Q2) – the middle value of the dataset

Third Quartile (Q3) – the value below which 75% of the data lies

Maximum value – the largest value in the dataset

The box in the plot represents the range from Q1 to Q3, known as the interquartile range (IQR). The line inside the box represents the median of the data. The whiskers extend from the box to show the minimum and maximum values, and any points outside this range are considered outliers.

Thus, a boxplot is useful for summarizing data distribution, comparing datasets, and detecting outliers in the data.

2 :- What information do we get from a box plot?

Ans :- A box plot provides a visual summary of the distribution of a dataset. It helps in understanding several important statistical properties of the data.

From a box plot, we can obtain the following information:

Median (Middle Value): The line inside the box represents the median, which shows the central value of the dataset.

Quartiles (Q1 and Q3): The bottom and top of the box represent the first quartile (Q1) and third quartile (Q3). These values divide the data into four equal parts.

Interquartile Range (IQR): The length of the box represents the interquartile range (Q3 – Q1), which shows the spread of the middle 50% of the data.

Minimum and Maximum Values: The whiskers extend from the box to show the smallest and largest values within the dataset.

Outliers: Any data points that fall outside the whiskers are shown separately and are called outliers.

Thus, a box plot helps us understand the center, spread, variability, and presence of outliers in the datasets

3 :- How is minimum and maximum whisker's marked in a box plot ?

In a box plot, the minimum and maximum whiskers are shown using lines that extend from the box to the smallest and largest data values within a certain range.

The minimum whisker extends from the first quartile (Q1) down to the smallest data value that is not considered an outlier.

The maximum whisker extends from the third quartile (Q3) up to the largest data value that is not considered an outlier.

These whiskers are usually drawn as thin lines with small horizontal caps at the ends, indicating the minimum and maximum values within the acceptable range.

If there are data values beyond these whiskers, they are displayed separately as individual points called outliers.

Thus, the minimum and maximum whiskers in a box plot mark the lowest and highest non-outlier values of the dataset.

4 :- How to we identify that some points in data are outliers by viewing the box plot?

Ans :- In a box plot, outliers can be identified by observing the data points that lie outside the whiskers of the plot.

The whiskers usually extend up to 1.5 times the interquartile range (IQR) from the first quartile (Q1) and the third quartile (Q3). Any data values that fall below Q1 – 1.5 × IQR or above Q3 + 1.5 × IQR are considered outliers.

These outliers are typically displayed as individual dots or small circles outside the whiskers in the box plot.

Thus, if we see points plotted separately beyond the whiskers, we can identify them as outliers in the dataset.

5 :- Read a data file from your drive and for all numeric columns plot boxplot.

```
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import drive

# Mount Google Drive
drive.mount('/content/drive')

# Read dataset
data = pd.read_csv('/content/drive/MyDrive/data.csv')

# Select numeric columns
numeric_data = data.select_dtypes(include='number')

# Plot boxplot
numeric_data.plot(kind='box', figsize=(10,6))

plt.title("Boxplot for Numeric Columns")
plt.show()
```

Explanation:

pd.read_csv() → reads the dataset from the drive.

select_dtypes() → selects only numeric columns (integer and float).

plot(kind='box') → creates boxplots for all numeric columns.

plt.show() → displays the boxplot.

Conclusion

Thus we have learned about basics of Contour plots, Histograms, Box plots.